

# Проектирование информационных систем

CASE-средства и  
Unified modeling language (UML)

# CASE-технология

- CASE (*Computer Aided Software/System Engineering*) технология – совокупность методологий анализа, проектирования, разработки и сопровождения ПО и комплекса взаимосвязанных программных средств автоматизации.
- В основе любой CASE-технологии лежит парадигма «методология-метод-нотация-средство».

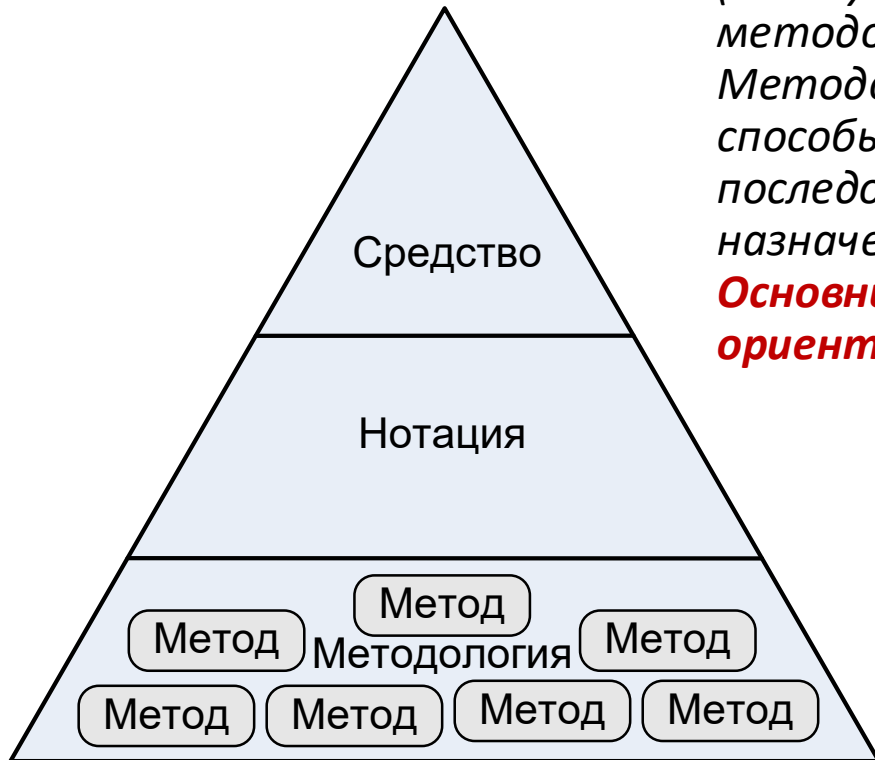
# CASE-технология. Парадигма «методология-метод-нотация-средство»

**Методология (или подход)** – система (совокупность) основных принципов, методик и методов, предназначенных для создания ПО. Методология определяет действия и методы (как способы реализации действий), их последовательность, правила распределения и назначение.

**Основные методологии: структурная и объектно-ориентированная.**

**Метод** - систематизированная совокупность шагов для выполнения определенной задачи (действия) или достижения цели (Совокупность однородных методов принято называть подходом).

**Нотация** – система условных обозначений, используемая для описания некоторого класса моделей. Выделяют графические (модели в виде графов, диаграмм, таблиц, схем) и текстовые (описание на формальных и естественных языках) нотации.



# Основные методологии

## Структурная

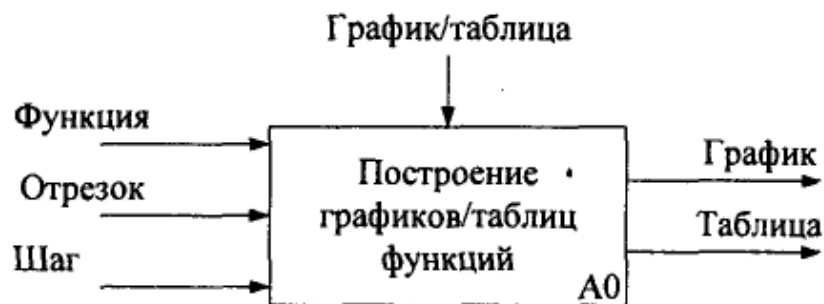
- В основе структурного подхода к разработке ПО лежит структурная декомпозиция – представление ПО (системы) в виде совокупности функциональных структур (подсистем, выполняющих автоматизируемые функции). То есть система разбивается на функциональные подсистемы, которые в свою очередь делятся на подфункции, подразделяемые на задачи и так далее.
- Процесс декомпозиции продолжается до получения конкретных процедур. При сохраняет целостное представление ПО (системы), в виде совокупности взаимодействующих функциональных компонентов (подсистем).
- Декомпозиция (анализ) - решение сложных задач путем их разделения на множество меньших, легких для понимания и решения.

## Объектно-ориентированная

- В основе объектного подхода к разработке программного обеспечения лежит объектная декомпозиция, т. е. представление разрабатываемого программного обеспечения в виде совокупности объектов, в процессе взаимодействия которых через передачу сообщений и происходит выполнение требуемых функций.

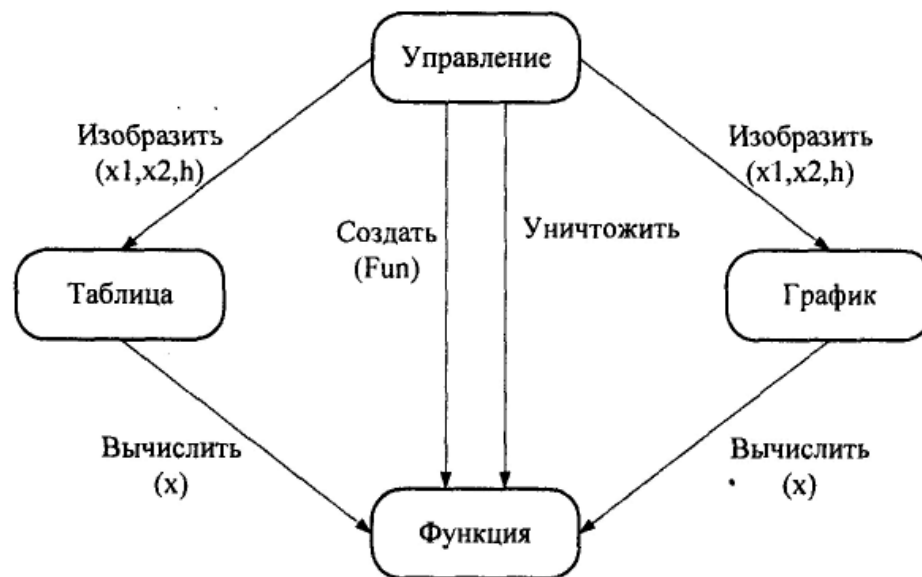
# Основные методологии

## Структурная



a

## Объектно-ориентированная



# Статистика результатов разработки ПО

- Исследования проектов разработки программного обеспечения показали, что во многих случаях **стоимость и длительность их реализации значительно превышали предполагаемые**, а характеристики качества не соответствовали требуемым, что наносило ущерб заказчикам, пользователям и разработчикам.
- По статистике
  - только **четверть** всех начатых проектов по разработке программных средств завершается своевременно,
  - **четверть** отменяется, и
  - около **половины** всех проектов завершается с превышением бюджетных затрат или с опозданием.

# Причины сложности разработки ПО (1)

- **Сложность реального мира.** Проблемы часто неизбежно содержат сложные элементы, а к соответствующим программам предъявляется множество различных, порой взаимоисключающих требований.
- **Трудности управления процессом разработки.** Основная задача разработчиков состоит в создании иллюзии простоты, в защите пользователей от сложности описываемого предмета или процесса. Сегодня обычными стали программные системы, размер которых исчисляется десятками тысяч или даже миллионами строк на языках высокого уровня. Ни один человек никогда не сможет полностью понять такую систему. Поэтому такой объем работ потребует привлечения команды разработчиков. Чем больше разработчиков, тем сложнее связи между ними и тем сложнее координация, особенно если участники работ географически удалены друг от друга.

# Причины сложности разработки ПО (2)

- **Гибкость программного обеспечения.** Программирование обладает предельной гибкостью. Такая гибкость чрезвычайно соблазнительна. Она заставляет разработчика создавать своими силами все базовые «строительные» блоки будущей конструкции. В отличие от строительной индустрии, где существуют единые стандарты на многие конструктивные элементы и качество материалов, в программной индустрии таких стандартов почти нет. Кроме того, часто приходится выполнять настройку программы под индивидуальные требования конкретного пользователя и системное окружение.
- **Проблема описания поведения больших дискретных систем.** Мы стараемся проектировать системы, разделяя их на части так, чтобы одна часть минимально воздействовало на другую. Всеобъемлющее тестирование таких программ провести невозможно. При неблагоприятных условиях небольшое внешнее событие может привести к критической ошибке системы.



# Пути борьбы со сложностью разработки ПО

- **Роль декомпозиции.** Как отмечает Дейкстра, "Способ управления сложными системами был известен еще в древности – divide et impera (разделяй и властвуй)".
- **Роль абстракции.** В экспериментах Миллера было установлено, что обычно человек может одновременно воспринять лишь  $7 \pm 2$  единицы информации. Люди развили чрезвычайно эффективную технологию преодоления сложности. Мы абстрагируемся от нее. Будучи не в состоянии полностью воссоздать сложный объект, мы просто игнорируем не слишком важные детали и, таким образом, имеем дело с обобщенной, идеализированной моделью объекта.
- **Роль иерархии.** Другим способом, расширяющим информационные единицы, является организация внутри системы иерархий классов и объектов.

# История развития ООП (1)

- Отдельные языки объектно-ориентированного моделирования стали появляться в период **между серединой 1970-х и концом 1980-х годов**, когда различные исследователи и программисты предлагали свои подходы к ООП.
- В период между **1989—1994** гг. общее число наиболее известных языков моделирования возросло с 10 до более чем 50.
- Многие пользователи испытывали серьезные затруднения при выборе языка ООП, поскольку ни один из них не удовлетворял всем требованиям, предъявляемым к построению моделей сложных систем.
- Принятие отдельных методик и графических нотаций в качестве стандартов (IDEF0, IDEF1X) не смогло изменить сложившуюся ситуацию непримиримой конкуренции между ними в начале 90-х годов, которая тоже получила название "**войны методов**".

# История развития ООП (2)

К середине 1990-х некоторые из методов были существенно улучшены и приобрели самостоятельное значение при решении различных задач ООАП. Наиболее известными в этот период становятся:

- **Метод Гради Буча** (Grady Booch), получивший условное название Booch или Booch'91, Booch Lite (позже — Booch'93), ориентирован на проектирование программных систем.
- **Метод Джеймса Румбаха** (James Rumbaugh), получивший название Object Modeling Technique - OMT (позже - OMT-2), ориентирован на анализ.
- **Метод Айвара Джекобсона** (Ivar Jacobson), получивший название Object-Oriented Software Engineering — OOSE, обеспечивает спецификацию бизнес-процессов и анализ требований при помощи сценариев использования.
- Усилия Г. Буча, Дж. Румбаха и А. Джекобсона привели к появлению первых документов, содержащих описание собственно языка **UML** версии 0.9 (июнь 1996 г.) и версии 0.91 (октябрь 1996 г.).

# История развития ООП (3)

- В этот период поддержка разработки языка UML становится одной из целей консорциума **OMG (Object Management Group)**. Хотя консорциум OMG был образован еще в 1989 году с целью разработки предложений по стандартизации объектных и компонентных технологий CORBA, язык UML приобрел статус второго стратегического направления в работе OMG.
- Именно в рамках OMG создается команда разработчиков под руководством Ричарда Соли, которая будет обеспечивать дальнейшую работу по унификации и стандартизации языка UML. В июне 1995 года OMG организовала совещание всех крупных специалистов и представителей входящих в консорциум компаний по методологиям ООП, на котором впервые в международном масштабе была признана целесообразность поиска индустриальных стандартов в области языков моделирования под эгидой OMG.
- В настоящее время все вопросы дальнейшей разработки языка UML сконцентрированы в рамках консорциума OMG.

# История развития ООП (4)

- В это же время стало ясно, что некоторые компании и организации видят в языке UML линию стратегических интересов для своего бизнеса. Компания **Rational Software** вместе с несколькими организациями, изъявившими желание выделить ресурсы для разработки строгого определения версии 1.0 языка UML, учредила **консорциум партнеров UML**, в который первоначально вошли такие компании, как Digital Equipment Corp., HP, i-Logix, Intellicorp, IBM, ICON Computing, MCI Systemhouse, Microsoft, Oracle, Rational Software, TI и Unisys. Эти компании обеспечили поддержку последующей работы по более точному и строгому определению нотации, что привело к появлению **версии 1.0 языка UML**.
- Компания Rational Software Corporation разработала и выпустила в продажу одно из первых инструментальных **CASE-средств Rational Rose 98**, в котором был реализован язык UML.
- **UML 2.4.1 принят в качестве международного стандарта [ISO/IEC 19505-1, 19505-2](#) (2011 г.).**

# CASE-средства

- CASE (Computer Aided Software/System Engineering) — набор инструментов и методов программной инженерии для создания программного обеспечения и анализа систем различной природы.

# Поколения CASE-средств

- **CASE-0:** средства анализа требований, проектирования спецификаций и структуры, редактирования программных интерфейсов; не поддерживали возможность создания графических моделей (**текстовые редакторы**);
- **CASE-1:** поддержка графических моделей (**визуальное моделирование**);
- Графические средства моделирования предметной области позволяют разработчикам в наглядном виде изучать существующую ИС, перестраивать ее в соответствии с поставленными целями и имеющимися ограничениями.
- **CASE-2:** **генерация исходных текстов; поддержка полного жизненного цикла разработки программного обеспечения** (возможность контроля, анализа, связывания системной информации и информации по управлению процессом проектирования, построение прототипов и моделей системы, тестирование, верификацию и анализ сгенерированных программ).

# Ориентация CASE-средств на этапы жизненного цикла

- Средства анализа (Upper CASE), предназначенные для построения и анализа моделей предметной области (BPwin);
- Средства анализа и проектирования (Middle CASE), поддерживающие наиболее распространенные методологии проектирования и использующиеся для создания проектных спецификаций (**Designer/2000**, **Rational Rose**, **Enterprise Architect**), обеспечивающие разработку спецификаций компонентов и интерфейсов системы, архитектуры системы, алгоритмов и структур данных;
- Средства проектирования баз данных, обеспечивающие моделирование данных и генерацию схем баз данных (как правило, на языке SQL) для наиболее распространенных СУБД (**Erwin**);
- Средства разработки приложений (Delphi);
- Вспомогательные типы включают:
  - средства планирования и управления проектом (**Microsoft Project** и др.);
  - средства конфигурационного управления (**PVCS** (Intersolv), **ClearCase**);
  - средства тестирования (**Quality Works** (Segue Software), **RationalRobot**);
  - средства документирования (**SoDA** (Rational Software)).



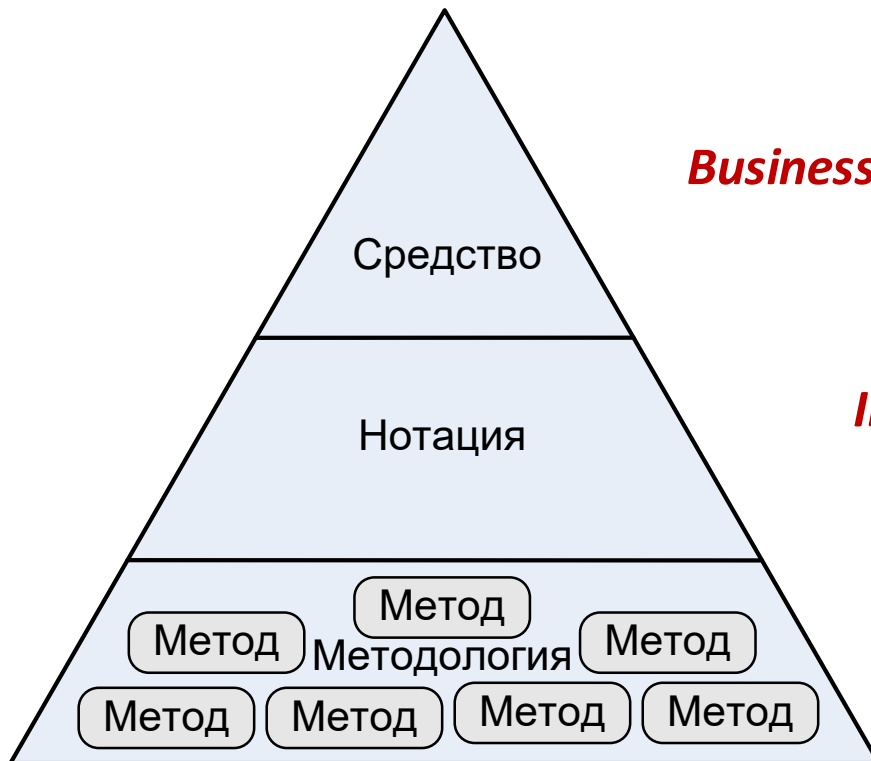
# *Сравнение ручного и автоматизированного создания ПО*

| Разработка без CASE-средств                              | Разработка с CASE-средствами                                       |
|----------------------------------------------------------|--------------------------------------------------------------------|
| Основные усилия направлены на кодирование и тестирование | Основные усилия направлены на анализ и проектирование              |
| «Бумажные» спецификации                                  | Быстрое итерационное прототипирование (частичное прототипирование) |
| Ручное кодирование                                       | Автоматизированная генерация кодов                                 |
| Ручное документирование                                  | Автоматическая генерация документации                              |
| Тестирование кодов                                       | Автоматический контроль проекта                                    |
| Сопровождение кодов                                      | Сопровождение спецификаций проектирования                          |

# *Сравнение ручного и автоматизированного создания ПО*

| Способ разработки | Трудозатраты этапа разработки, % |                |             |              |
|-------------------|----------------------------------|----------------|-------------|--------------|
|                   | Анализ                           | Проектирование | Кодирование | Тестирование |
| без CASE-средств  | 20                               | 15             | 20          | 45           |
| CASE-средства     | 40                               | 40             | 5           | 15           |

# **CASE Computer Aided Software/System Engineering -** **технология. Парадигма «методология- метод-нотация-средство»**



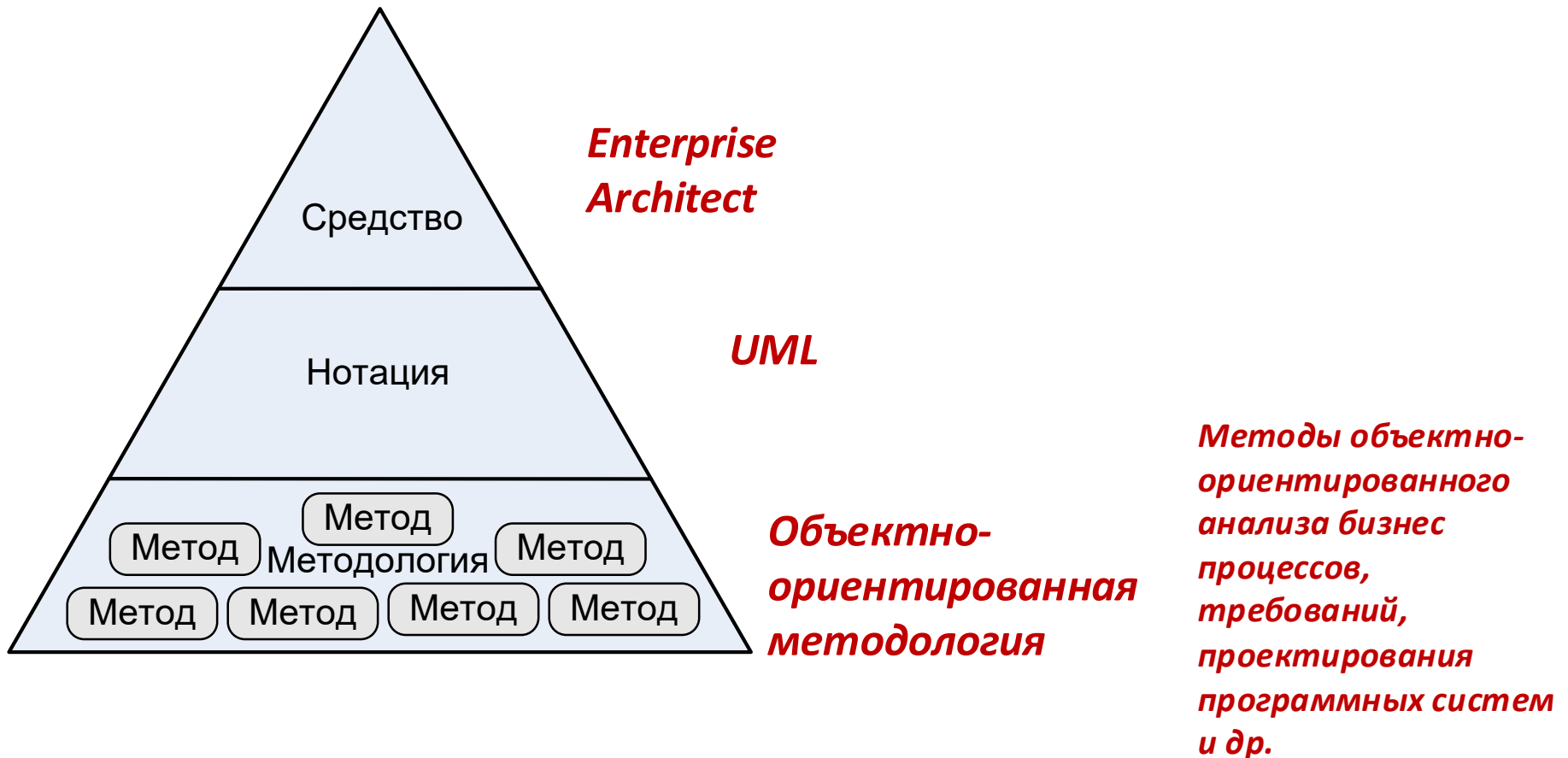
***Business Studio/Ramus***

***IDEF***

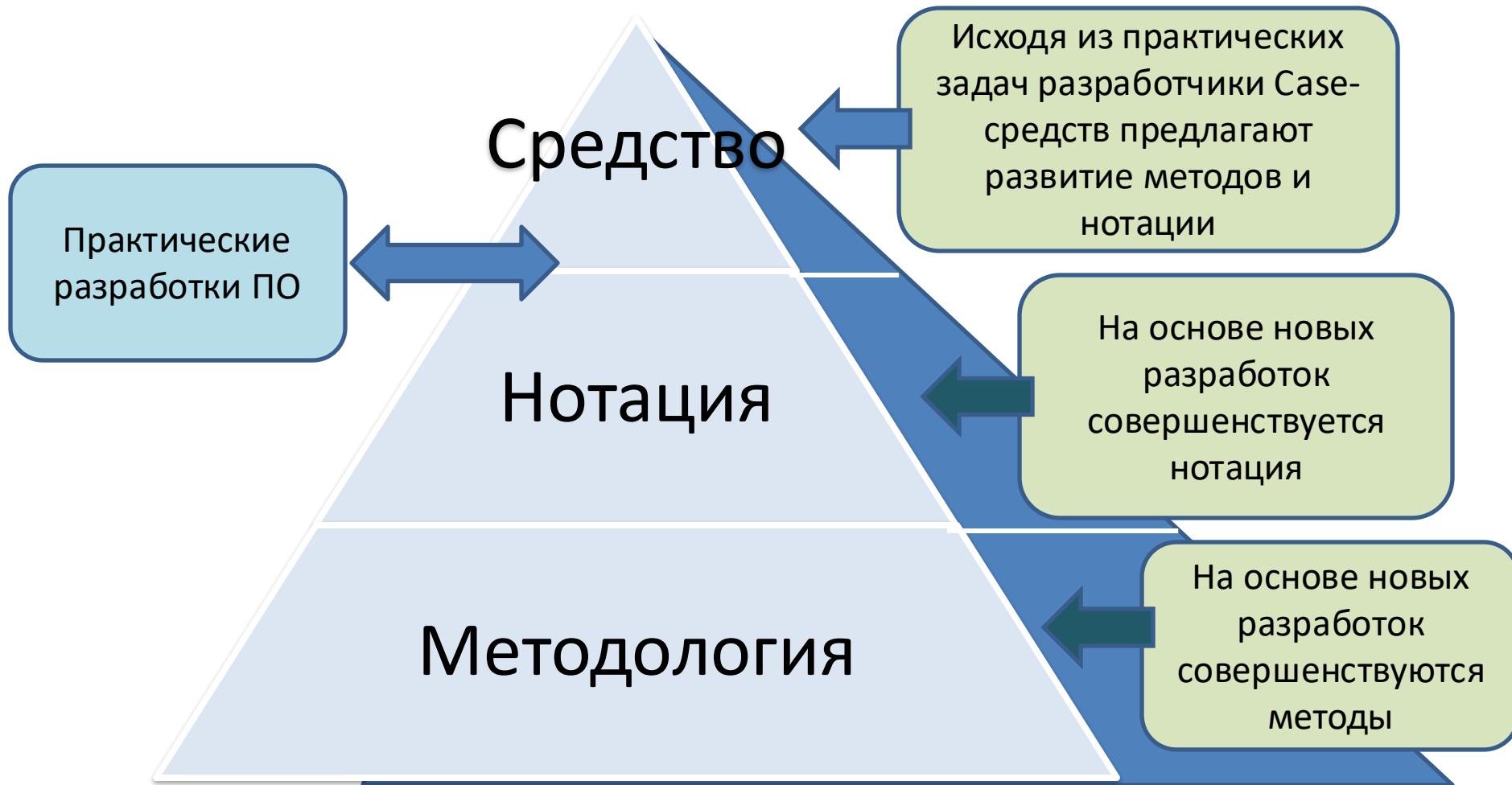
***Структурно-ориентированная методология***

***Методы структурно-ориентированного анализа бизнес процессов, требований, проектирования программных систем и др.***

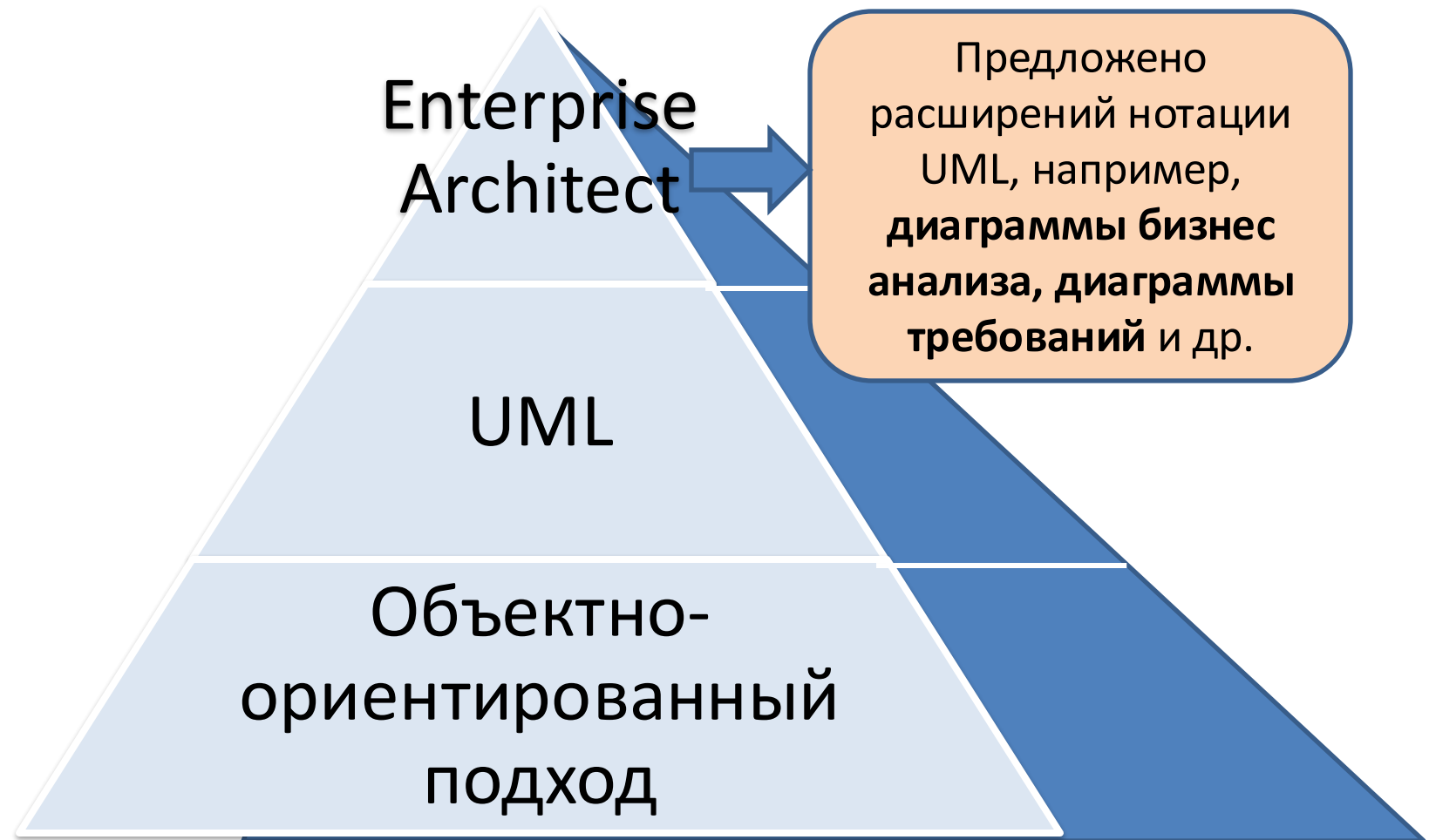
# **CASE Computer Aided Software/System Engineering - технология. Парадигма «методология- метод-нотация-средство»**



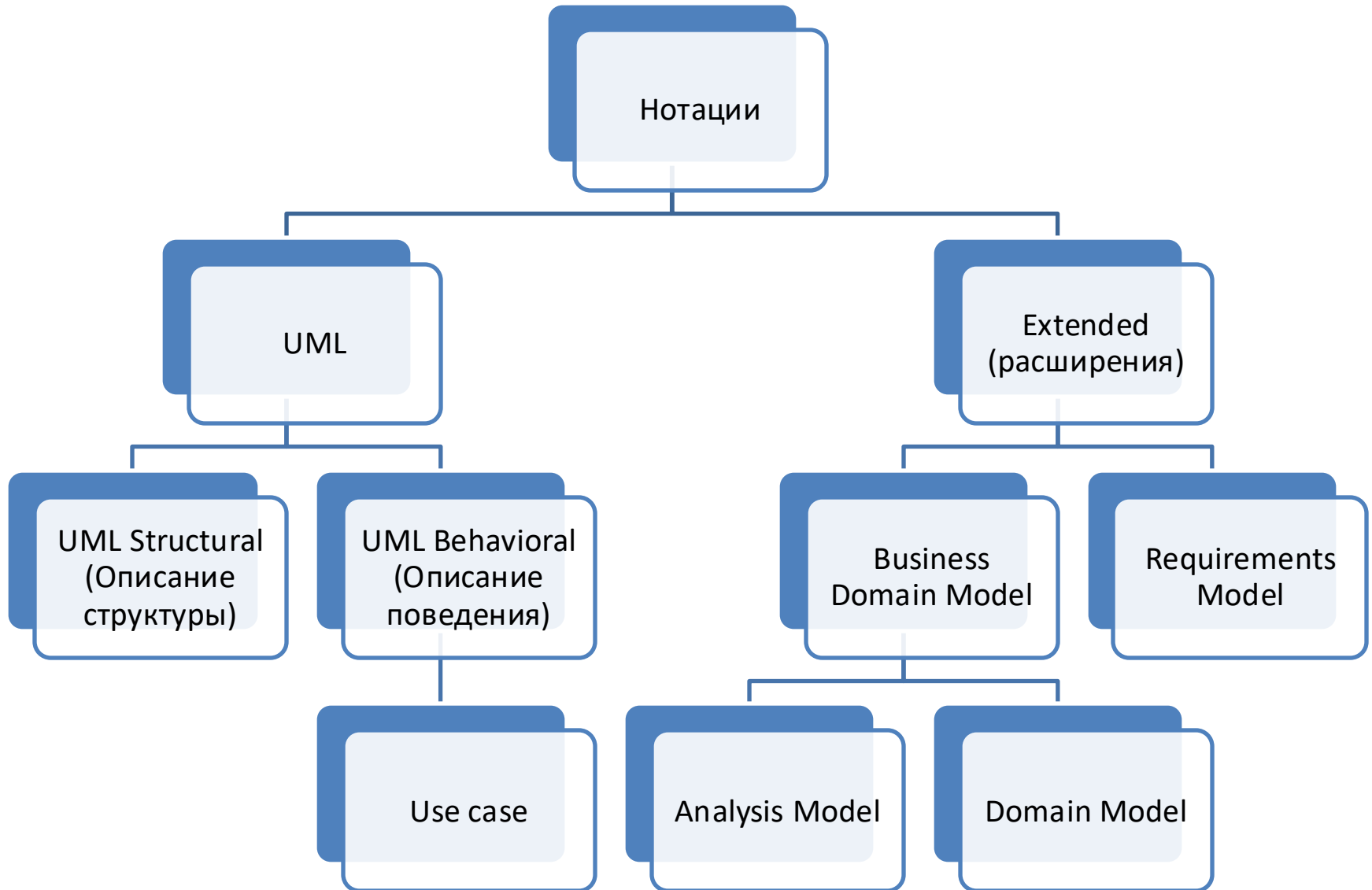
# Особенности CASE *Computer Aided Software/System Engineering* - технологии



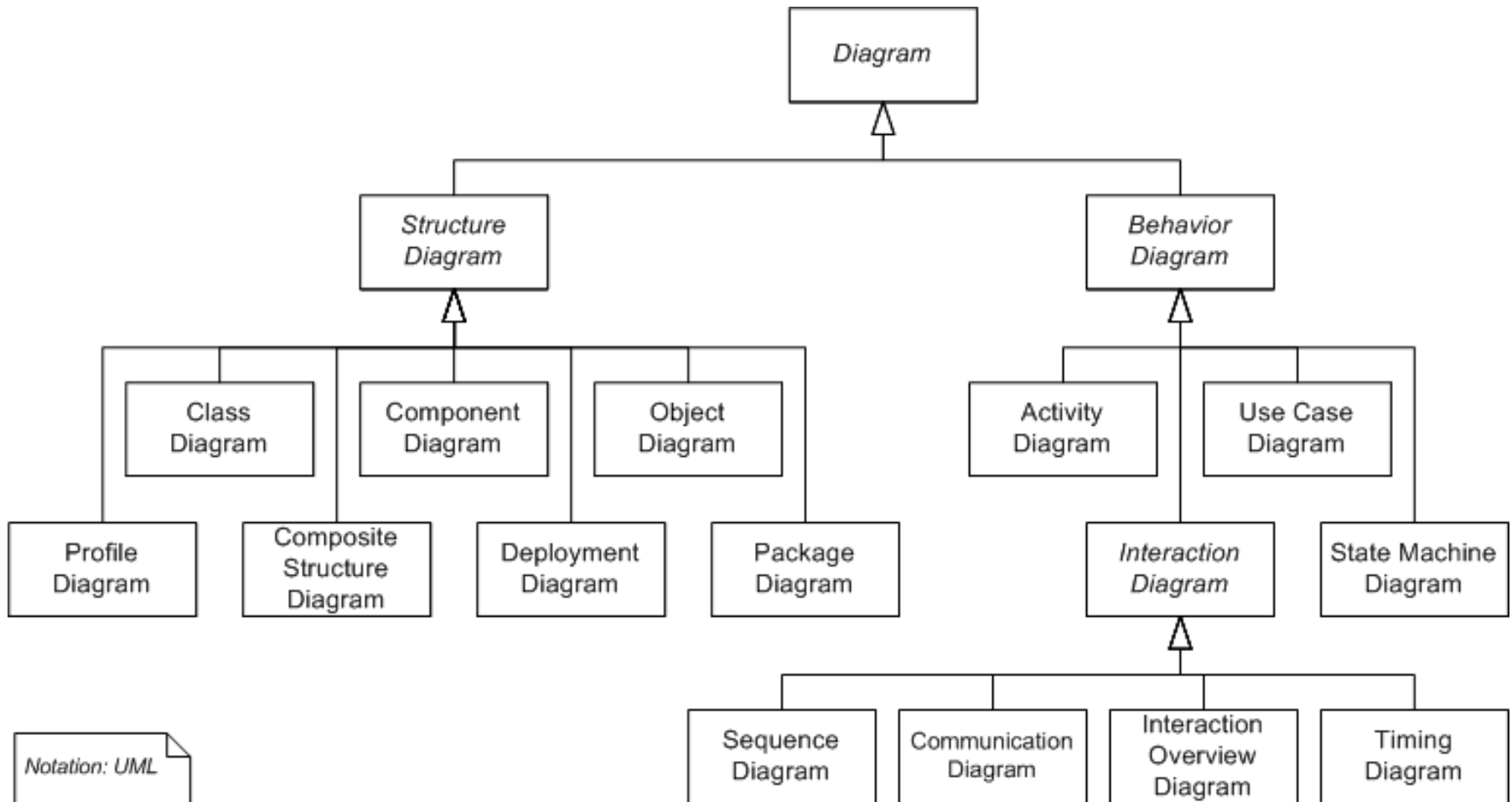
# Особенности CASE *Computer Aided Software/System Engineering* - технологии



# Фрагмент структуры нотаций ЕА

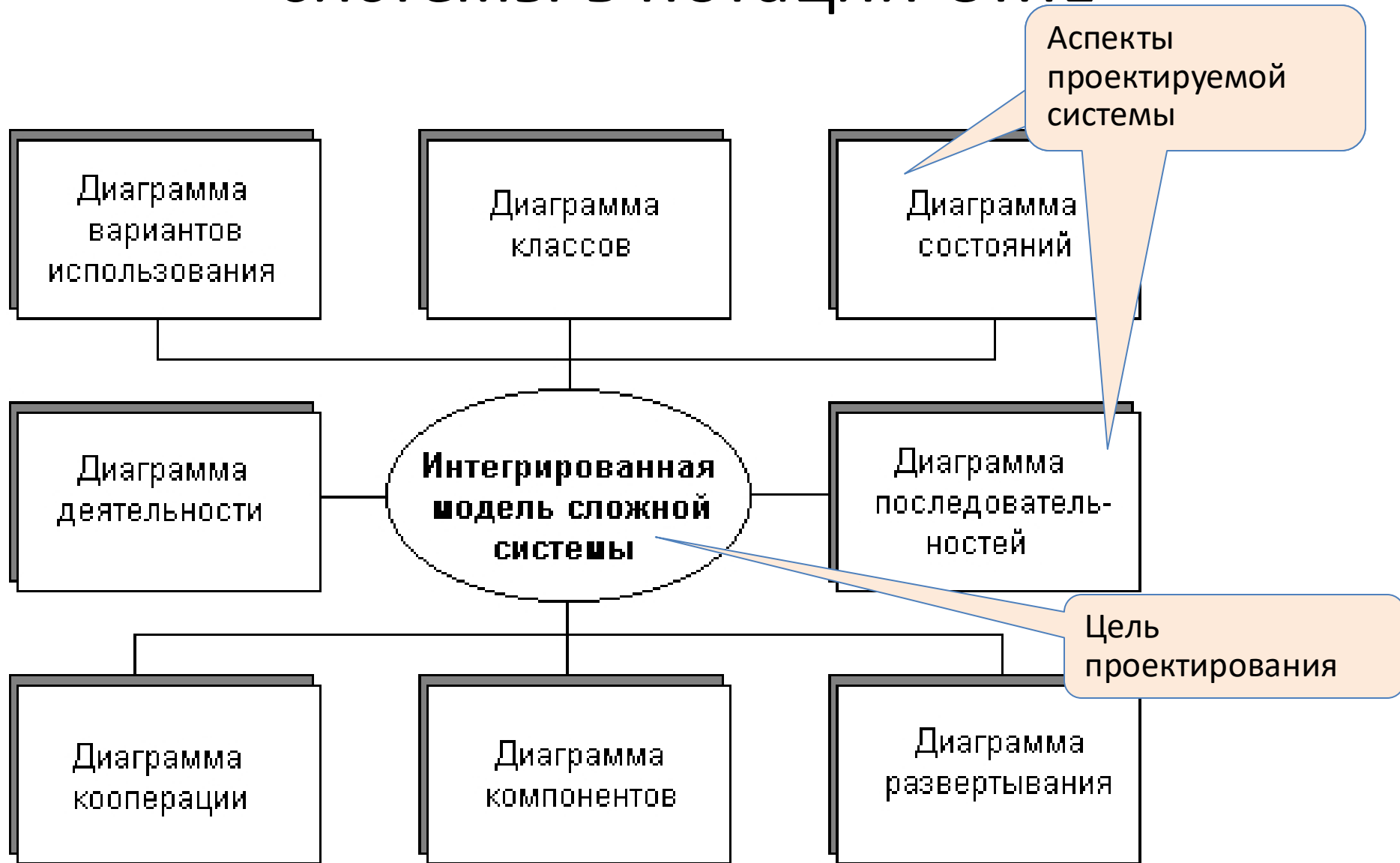


# Фрагмент структуры нотаций UML





# Интегрированная модель сложной системы в нотации UML



# Жизненный цикл ПО.

## Этапы проектирования ПО

- **Постановка задачи** (стадия «Техническое задание»);
- **Анализ требований и разработка спецификаций** (стадия «Эскизный проект»);
- **Проектирование** (стадия «Технический проект»);
- **Реализация** (стадия «Рабочий проект (рабочая документация)»);
- **Внедрение** («Ввод в действие»)
- **Сопровождение** («Сопровождение»).

# Методы и нотации в рамках этапов ЖЦ ПО (на примере UML)

Постановка задачи  
(«Техническое  
задание»)

Анализ требований и  
разработка  
спецификаций  
(«Эскизный проект»)

Analysis Model

Use case

Requirements Model

Domain Model

Interface

Проектирование  
(«Технический  
проект»)

Class

State

Activity

Sequence

Communication

Component

Deployment

# Основные виды графических конструкций UML

- **Значки или пиктограммы.** Значок представляет собой графическую фигуру фиксированного размера и формы. Она не может увеличивать свои размеры, чтобы разместить внутри себя дополнительные символы. Значки могут размещаться как внутри других графических конструкций, так и вне их. Примерами значков могут служить окончания связей элементов диаграмм или некоторые другие дополнительные обозначения (украшения).
- **Графические символы на плоскости.** Такие двумерные символы изображаются с помощью некоторых геометрических фигур и могут иметь различную высоту и ширину с целью размещения внутри этих фигур других конструкций языка UML. Наиболее часто внутри таких символов помещаются строки текста, которые уточняют семантику или фиксируют отдельные свойства соответствующих элементов языка UML. Информация, содержащаяся внутри фигур, имеет важное значение для конкретной модели проектируемой системы, поскольку регламентирует реализацию соответствующих элементов в программном коде.

# Основные виды графических конструкций UML

- **Пути, которые представляют собой последовательности из отрезков линий, соединяющих отдельные графические символы.** При этом концевые точки отрезков линий должны обязательно соприкасаться с геометрическими фигурами, служащими для обозначения вершин диаграмм, как принято в теории графов. Другими словами, пути не могут обрываться на диаграмме линией, которая не соприкасается ни с одним графическим символом. Как отмечалось выше, пути могут иметь в качестве окончания или терминатора специальную графическую фигуру — значок, который изображается на одном из концов линий, являющихся сегментами этого пути.
- **Строки текста.** Служат для представления различных видов информации в некоторой грамматической форме. Предполагается, что каждое использование строки текста должно соответствовать синтаксису в нотации языка UML, посредством которого может быть реализован грамматический разбор этой строки. Последний необходим для получения полной информации о модели. Например, строки текста в различных секциях обозначения класса могут соответствовать атрибутам этого класса или его операциям.

# Рекомендации по созданию диаграмм

- **Каждая диаграмма должна служить законченным представлением** соответствующего фрагмента моделируемой предметной области. Отсутствие тех или иных элементов на диаграмме служит признаком неполноты модели и может потребовать ее последующей доработки.
- **Все сущности на диаграмме модели должны быть одного концептуального уровня.** Здесь имеется в виду согласованность не только имен одинаковых элементов, но и возможность вложения отдельных диаграмм друг в друга для достижения полноты представлений. В случае достаточно сложных моделей систем желательно придерживаться стратегии последовательного уточнения или детализации отдельных диаграмм.
- **Вся информация о сущностях должна быть явно представлена на диаграммах.** Необходимо стремиться к явному указанию свойств всех элементов диаграмм.

# Рекомендации по созданию диаграмм

- **Диаграммы не должны содержать противоречивой информации.** Противоречивость модели может служить причиной серьезных проблем при ее реализации и последующем использовании на практике. Например, наличие элементов с одинаковыми именами и различными атрибутами свойств в одном пространстве имен также приводит к неоднозначной интерпретации и может служить источником проблем.
- **Диаграммы не следует перегружать текстовой информацией.** Как правило, наличие больших фрагментов развернутого текста служит признаком недостаточной проработанности модели.

# Рекомендации по созданию диаграмм

- **Каждая диаграмма должна быть самодостаточной** для правильной интерпретации всех ее элементов и понимания семантики всех используемых графических символов. *Любые пояснительные тексты, которые не являются собственными элементами диаграммы (например, комментариями), не должны приниматься во внимание разработчиками.* В то же время отдельные достаточно общие фрагменты диаграмм могут уточняться или детализироваться на других диаграммах этого же типа, образуя вложенные или подчиненные диаграммы. Таким образом, модель системы на языке UML представляет собой пакет иерархически вложенных диаграмм, детализация которых должна быть достаточной для последующей генерации программного кода.
- **Количество типов диаграмм для конкретной модели приложения не является строго фиксированным.** Речь идет о том, что для простых приложений нет необходимости строить все без исключения типы диаграмм. Некоторые из них могут просто отсутствовать в проекте системы, и этот факт **не будет считаться ошибкой разработчика.** Например, модель системы может не содержать диаграмму развертывания для приложения, выполняемого локально на компьютере пользователя. Важно понимать, что перечень диаграмм зависит от специфики конкретного проекта системы.